

Exercise :Fetching data takes some amount of time. A user has to wait until the data get loaded. Implement a loading functionality of a data is not fetched yet. You can simulate the delay using setTimeout. Use state from class component react to do this exercise

JavaScript

```
import React from 'react';

class DataFetcher extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      data: null,
      loading: true, // Inicialmente, o componente está carregando
      error: null,
    };
  }

  componentDidMount() {
    // Simula uma chamada de API ou outra operação assíncrona
    setTimeout(() => {
      // Simula dados sendo recebidos (substitua por sua lógica real
      de busca de dados)
      const simulatedData = {
        name: "John Doe",
        age: 30,
        city: "New York",
      };

      // Simula um erro (comente ou remova conforme necessário)
      // const simulatedError = new Error("Failed to fetch data");

      if (simulatedData) {
        this.setState({ data: simulatedData, loading: false });
      } else if (simulatedError) {
        this.setState({ error: simulatedError, loading: false });
      } else { //caso não tenha dados nem erro
        this.setState({ loading: false });
      }
    }, 2000);
  }
}
```

```

    },
    }, 2000); // Simula um atraso de 2 segundos
  }

  render() {
    const { data, loading, error } = this.state;

    if (loading) {
      return <div>Loading...</div>; // Mostra uma mensagem de
      carregamento
    }

    if (error) {
      return <div>Error: {error.message}</div>; // Mostra uma mensagem
      de erro
    }

    if (!data) {
      return <div>No data available.</div>; // Mensagem se não houver
      dados
    }

    return (
      <div>
        <h2>User Data:</h2>
        <p>Name: {data.name}</p>
        <p>Age: {data.age}</p>
        <p>City: {data.city}</p>
      </div>
    );
  }
}

export default DataFetcher;

```

Explanation:

1. State:

- data: Stores the fetched data (initially null).
- loading: A boolean indicating whether the data is being fetched (initially true).
- error: Stores any error that occurs during data fetching (initially null).

2. componentDidMount():

- This lifecycle method is called after the component is rendered for the first time. It's the ideal place to perform data fetching.
- setTimeout() simulates an asynchronous operation (like an API call). Replace this with your actual data fetching logic (e.g., using fetch or axios).
- Inside setTimeout():

- Simulated data (simulatedData) is created. Replace this with the data you receive from your API.
- The setState() method is used to update the component's state:
 - If data is successfully fetched, data is set to the fetched data, and loading is set to false.
 - If an error occurs, error is set to the error object, and loading is set to false.
 - Se não houver dados e nem erro loading é setado para false.

3. render():

- The render() method checks the loading state:
 - If loading is true, it renders a "Loading..." message.
 - If error is not null, it renders an error message.
 - If data is not null, it renders the fetched data.
 - Se não houver dados, exibe a mensagem "No data available".

How to use:

1. Import the DataFetcher component into your desired component or page.
2. Render the DataFetcher component:

JavaScript

```
import DataFetcher from './DataFetcher'; // Adjust path as needed

function MyComponent () {
  return (
    <div>
      <DataFetcher />
    </div>
  );
}
```

Now, when the component mounts, it will simulate fetching data. While the data is being fetched, you'll see the "Loading..." message. Once the simulated data is "received," the component will re-render and display the data. If there's an error, it will show the error message. If there's no data, it will display the message "No data available.". Remember to replace the simulated data and the setTimeout with your actual data fetching logic.